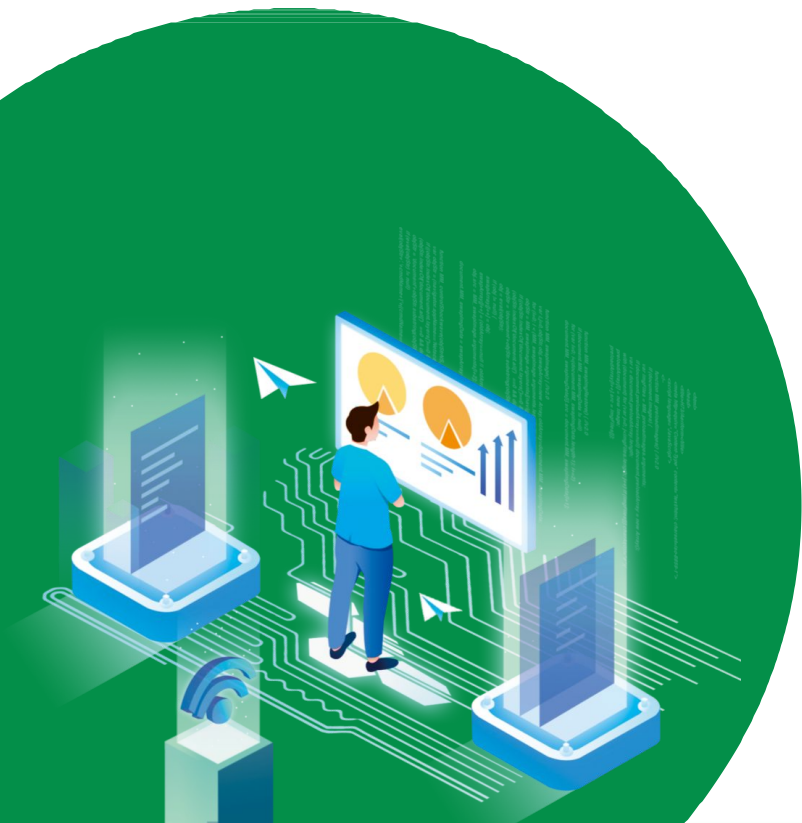


Performance Testing & Observability

Presenter: [Link\(a\)](#)



CONTENT

01. Performance Testing

02. Observability & Monitoring

1

Performance testing



**Performance Testing
Overview**



Tool & Practice



Performance Testing

Performance testing is a **non-functional software testing** technique that determines how the **stability, speed, scalability, and responsiveness** of an application holds up under a **given workload**.

Functional requirements:

- Authentication & Authorization
- Data processing & Business logic
- APIs & Integrations
- Data consistency & Transactions,...

Non-functional requirement:

- Performance
- Scalability
- Low latency (P99, P95)
- High availability (SLA four nights, five nights,...)

Back-of-the-envelope estimation:

- Daily active users
- QPS
- Read/write throughput



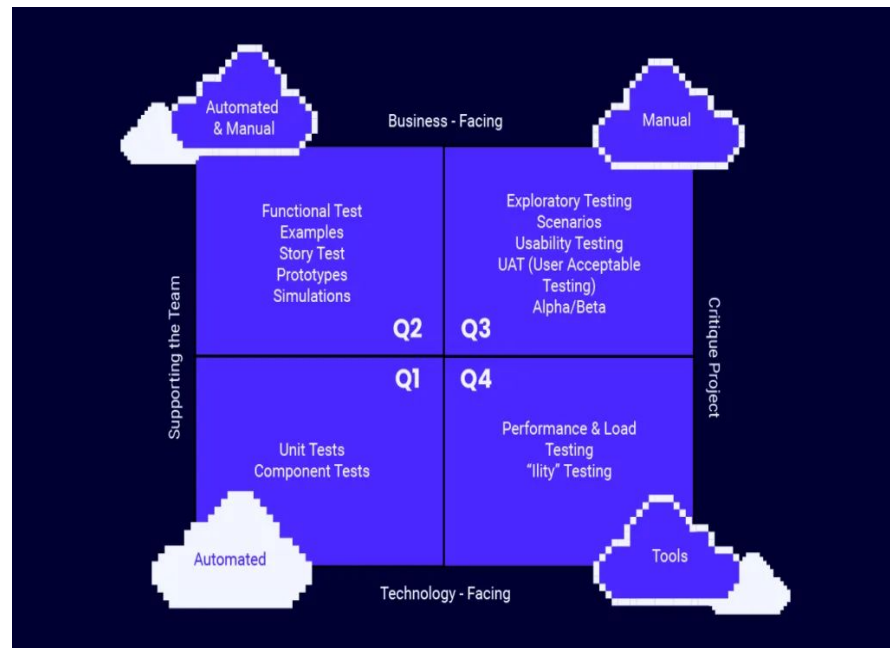
How/When Do Performance Testing



It's a form of **non-functional testing**, meaning that it's largely unconcerned with the user interface.

It depend on the software and methodology the organization is using:

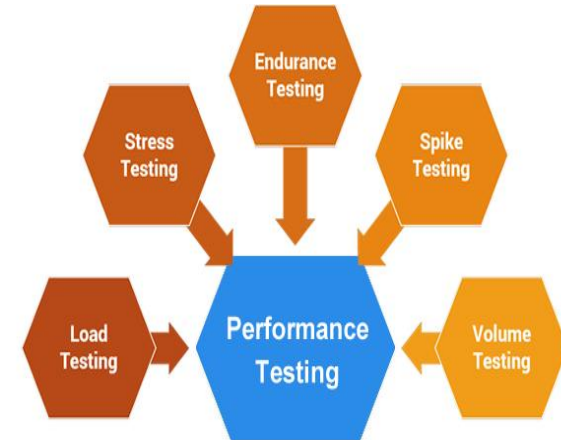
- **Waterfall**: after development has finished
- **Agile**: likely happen throughout the development process or integrated with the CI tool.
- **Campaign event, locate computing bottlenecks, compare two or more systems,..**





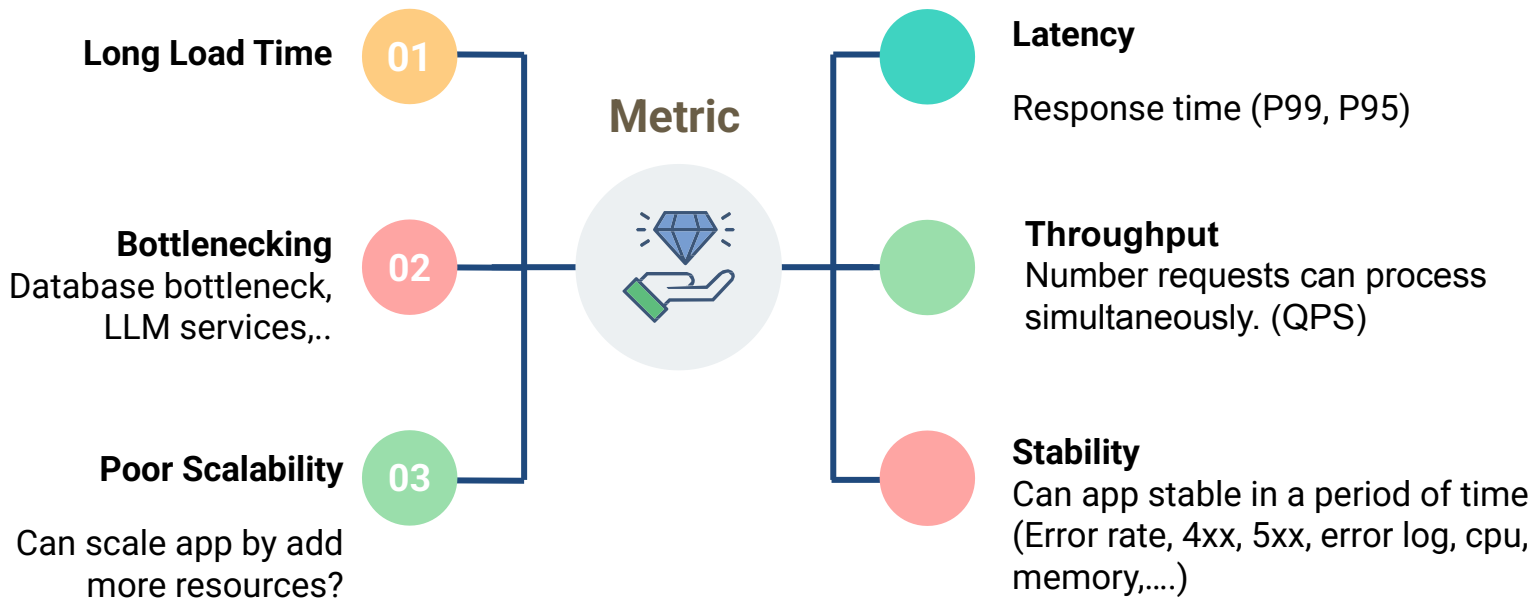
Types of performance testing

- **Load Test:** Test how the system responds to real-world load conditions
- **Stress Test:** Focus on the ability of a system to handle peak loads that are at or beyond the limits of its anticipated or specified work- loads.
- **Endurance/Soak Test:** Simulates high traffic for an extended period of time. Checks the software's ability to tolerate extended periods of high traffic. (memory leaks, database connections, thread pools)
- **Spike Test:** Tests the system's ability to recover from a load spike. It focuses on the ability of a system to respond correctly to sudden bursts of peak loads and return afterwards to a steady state.
- **Volume Test:** Checks that the software can handle and process a **large amount of data** at once without breaking, slowing down, or losing any information.





Performance testing metrics

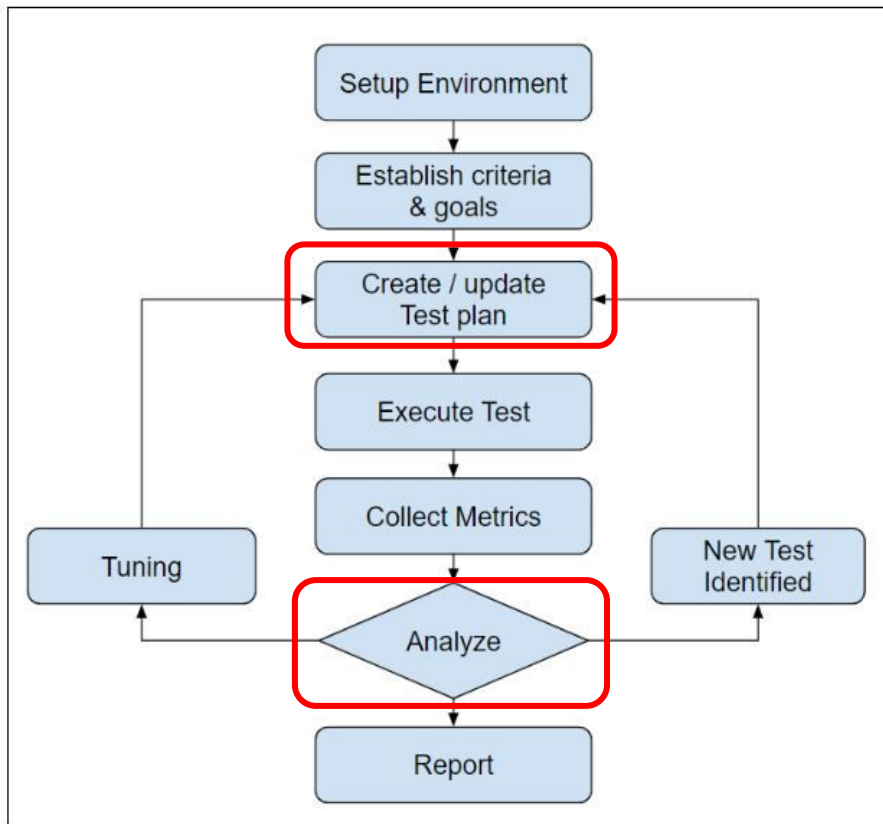




Performance testing steps

Main point:

- Create test plan
- Analyze report





Performance testing tools

Open Source

- **Jmeter**
- **Gatling**
- **Locust** (Python)
- k6
- Tsung
- Postman

Enterprise

- LoadRunner
- Neoload
- BlazeMeter
- AppDynamics
- Dynatrace
- WebLoad



Performance testing tools

	JMETER	LOCUST	GATLING (Open Source)
TEST SCENARIOS	No-Code / Low-Code	High-Code	High-Code
LICENSE	Fully Open Source	Fully Open Source	Partially Open Source
SCRIPT RECORDING	GUI	Manually	GUI
EXECUTION MONITORING	Console + Graphs	Console + Graphs	Console
REPORTING	ENOUGH	ENOUGH	ENOUGH
RESOURCES CONSUMPTION	HIGH MEMORY	HIGH CPU	MEDIUM USAGE
Community, Support, Documentation	HIGH	ENOUGH	ENOUGH
CI / CD Integration	POWERFUL	ENOUGH	ENOUGH
Learning Curve	EASY	MID	DIFFICULT

Beanshell / Java

Python

Scala (similar with Java)



Apache JMeter

Apache JMeter is an open-source, purely Java-based software, designed to load test functional behavior and measure performance. It was originally designed for testing Web Applications but has since expanded to other test functions.

Features:

- Ability to load and performance test many different applications/server/protocol types:
 - Web - HTTP, HTTPS, SOAP/REST
 - Database via JDBC
 - FTP, LDAP, MAIL (SMTP, POP3, IMAP)
 - TCP
- Full featured Test IDE that allows fast Test Plan, building and debugging
- Command-line mode to load test from any Java compatible OS (Linux, Windows, Mac,...)
- A complete and ready to present dynamic HTML report



Apache JMeter

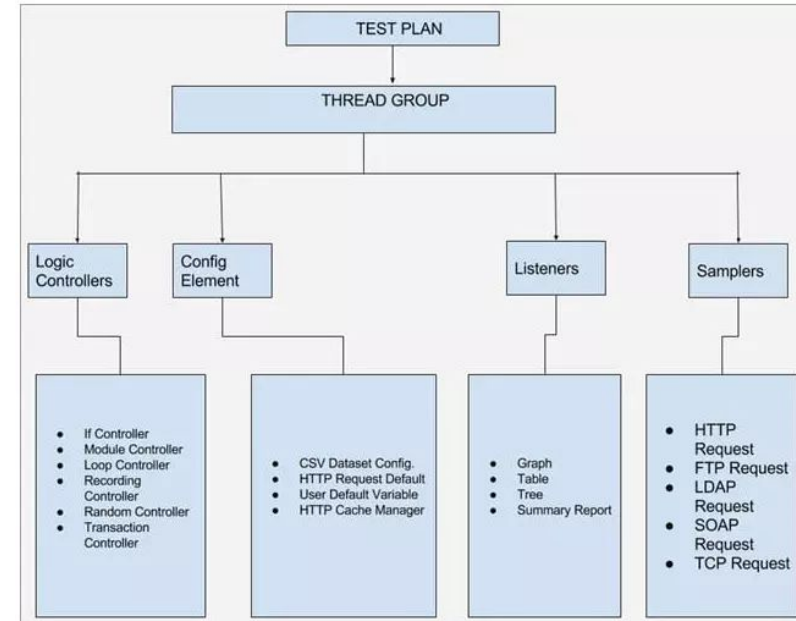
Usually follow this process:

- **Create test plan:** Run Jmeter in GUI mode and create specific plan, prepare test data (csv, xml, random data, Java code,...)
- **Load test running:** Run Jmeter in CLI mode, try to OS turning, machine size, increase Java heap size,...
- **Load test analysis:** check HTML report to analyze your load test



Apache JMeter: Create Test Plan

- **ThreadGroup (Users)**: setup number of threads (users), time to execute, plan to load test, stress test, spike test,...
- **Logic Controllers**: Customize the logic that JMeter uses to decide when to send requests.
- **Samplers**: config to send requests to a server and wait for a response.
- **Config Element**: works closely with a Sampler, it can add to or modify requests.
- **Listeners**: Shows details of sampler requests and responses

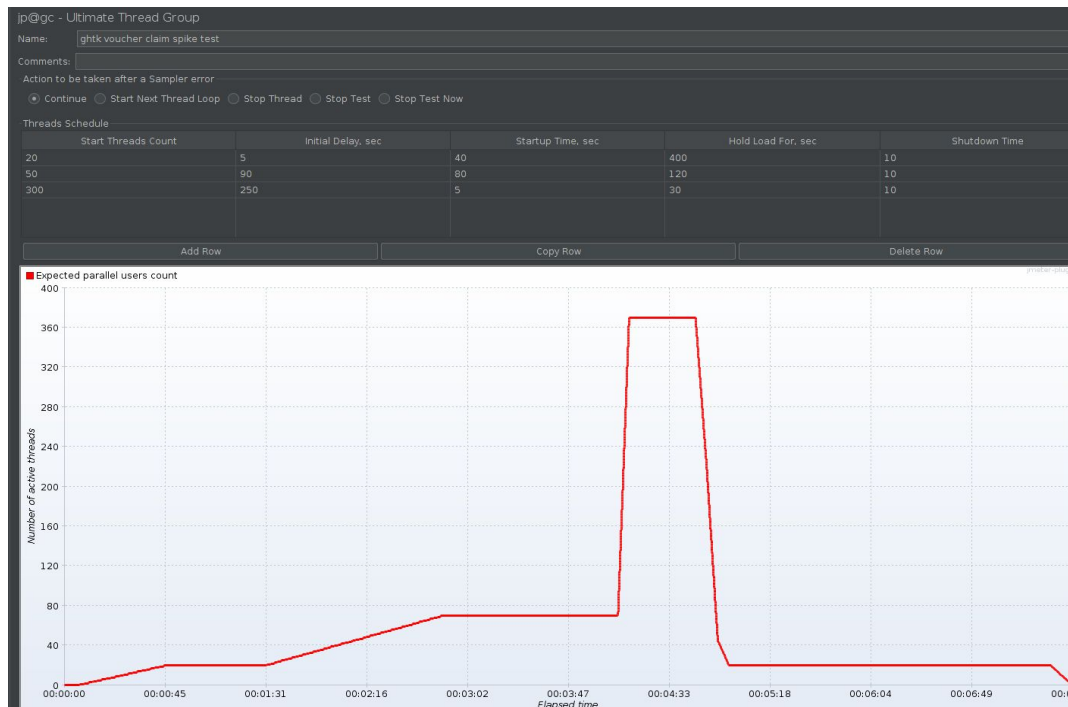




Apache JMeter: Build A Web Test Plan

Spike Test - create plan for voucher claim:

- 20 users active whole testing
- 50 users join after 90s and out after “gold time”
- 300 users join and claim voucher at “gold time”

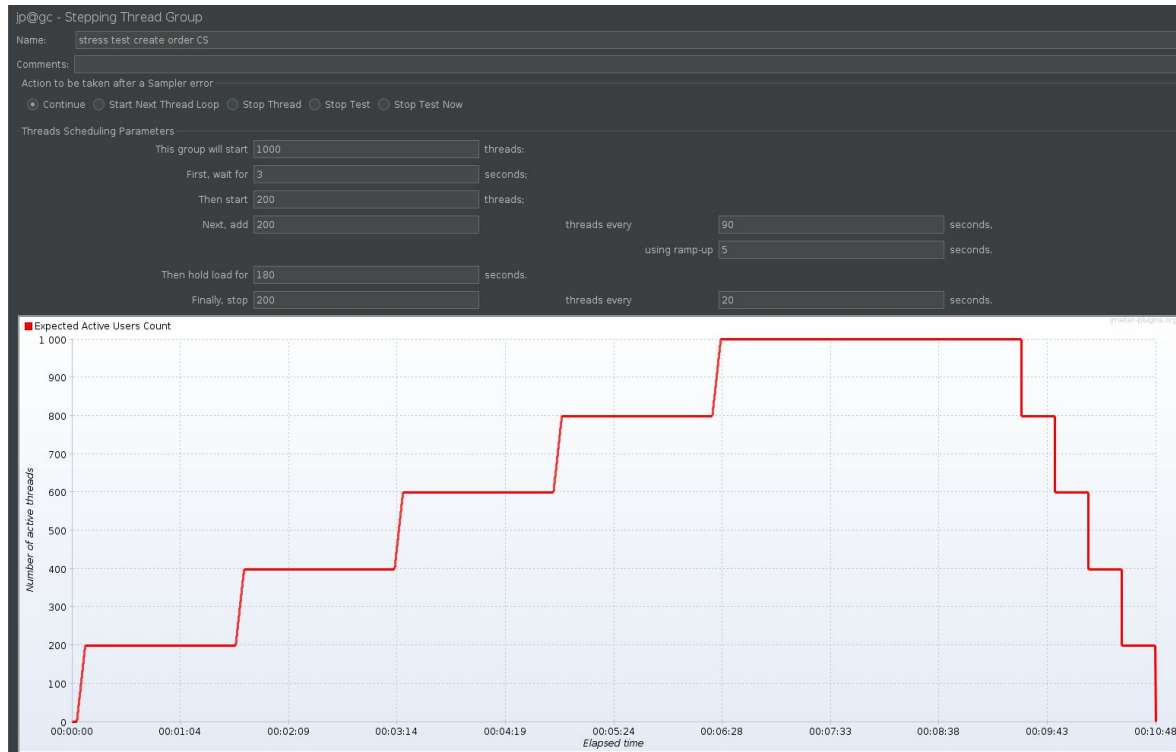




Apache JMeter: Build A Web Test Plan

Stress Test - Create order Tiktok campaign:

- Duration 10m
- Add 200 threads every 90s to find maximum CCU



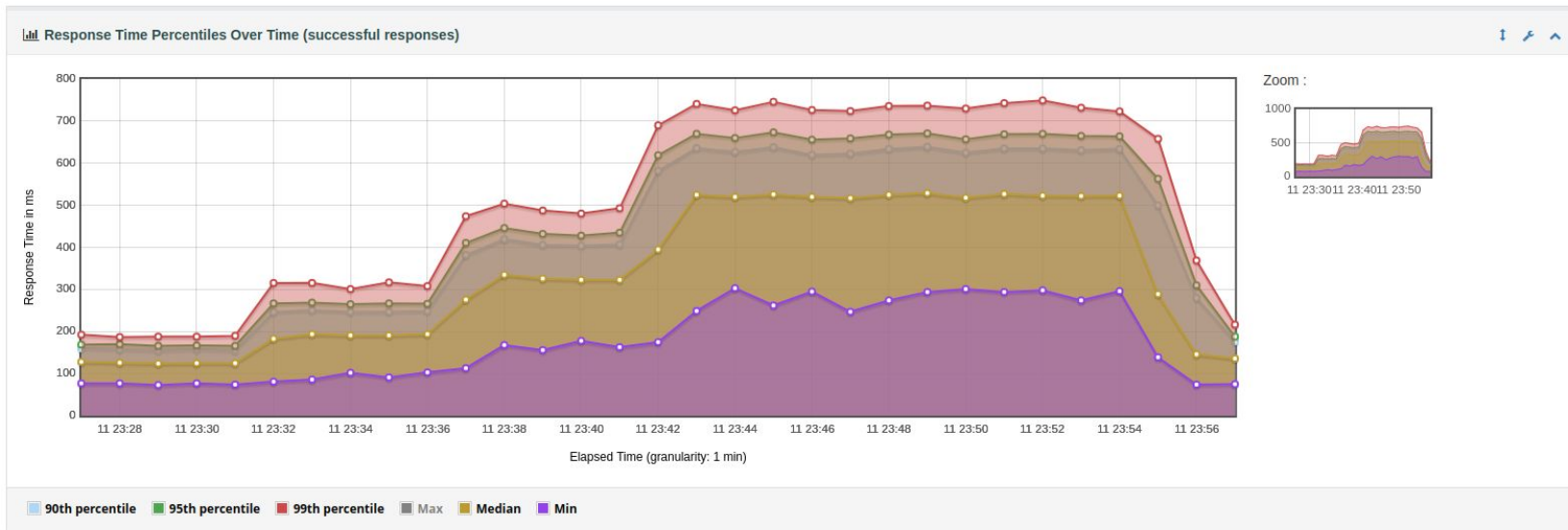


Apache JMeter: Output

Example report: Smart Tagger (AI Service):

https://docs.google.com/document/d/1fK7dezo4q1sMMLSi_dTlaHjGPeAFC698hDEz6byQ9bk

file:///Users/link/Downloads/result_stress_test/output_smart_tagger/index.html



2

Observability

y



Observability & Monitoring



Tool & Practice



Observability

Observability is the ability to measure the internal states of a system by examining its outputs.

- A system becomes '**observable**' when it is possible to estimate the current state using only information from outputs.
- Data from Observability can identify and troubleshoot problems, optimize performance, and improve security.

Observability

- Tells you why a system is at fault
- Acts as a knowledge base in defining what to monitor
- Focuses on giving context to the data
- Gives a more complete assessment of the overall environment
- Observability is a traversable map
- It gives you complete Information
- Observability creates the potential to monitor different events

VS

Monitoring

- Notifies you that a system is at fault
- Focuses on monitoring the systems & discovering faults
- Focuses on collecting data
- Focuses on monitoring KPIs
- Monitoring is a single plane
- It gives you limited Information
- Monitoring is the process of using observability



Observability

Metrics:

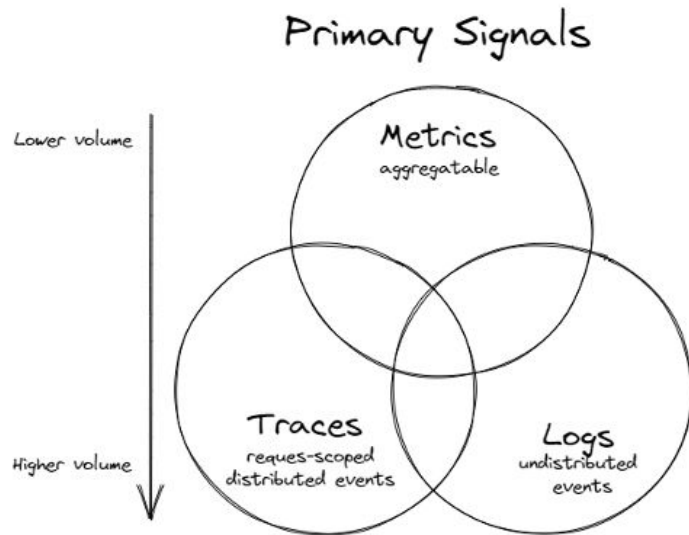
- Inform observers about the health & operations.
- Use to track the occurrence of an event, counting of items, time taken to perform an action, report current resource (CPU, memory,...)

Logs:

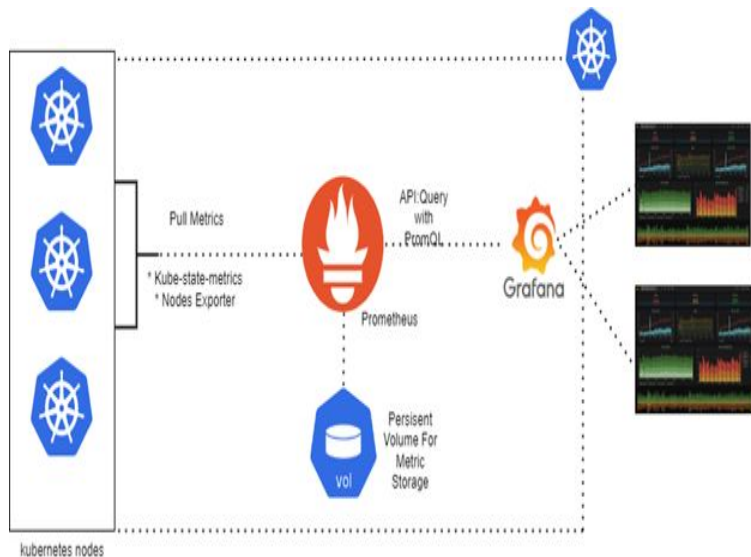
- Inform observers about the discrete events that occurred.
- Use to track detailed information about an event also monitored by a metric, particularly errors, warnings or other exceptional situations.

Traces:

- Inform a program's flow and data progression.
- Which function? The function's duration, parameters passed.
- Use to provides visibility into how a request is processed across multiple services.



Metrics



Advantages of Metrics

- Quantitative and intuitive for setting alert thresholds
- Lightweight and cost-effective for storage
- Excellent for tracking trends and system changes
- Provides real-time component state data
- Constant overhead cost; not affected by data surges

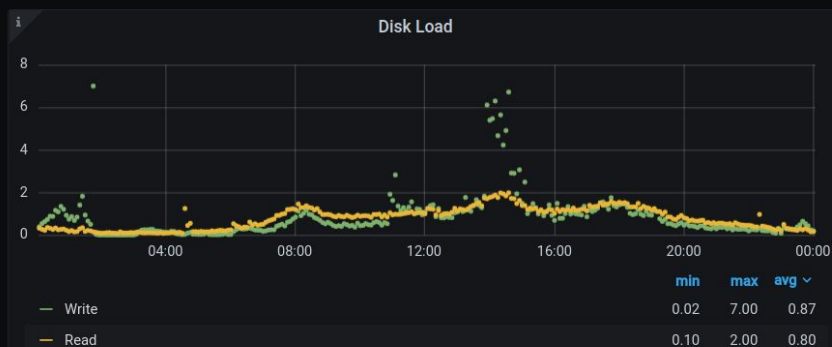
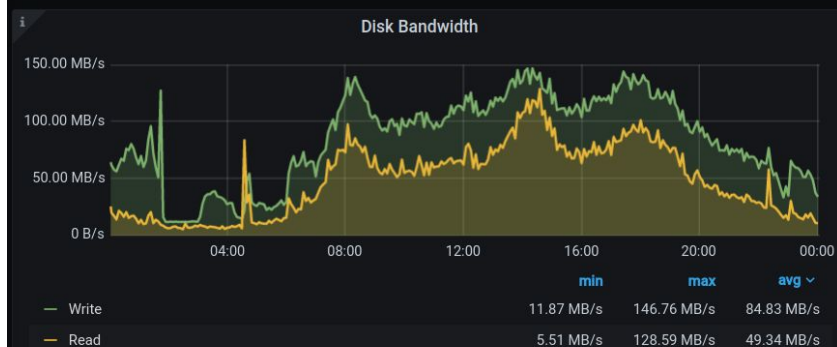
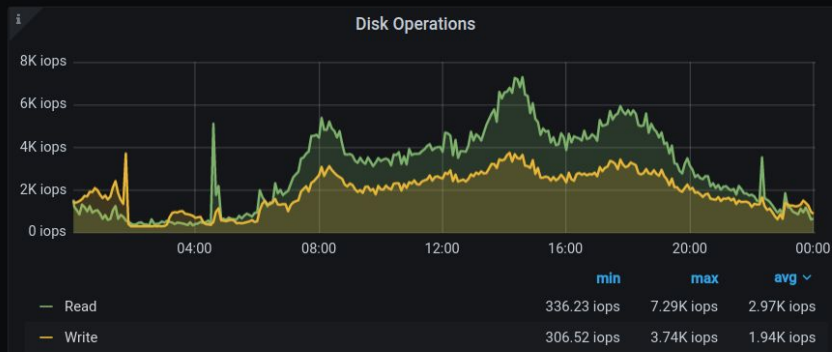
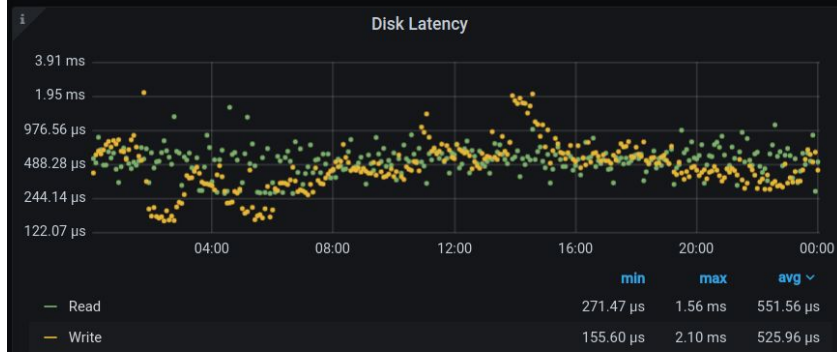
Challenges of Metrics

- Limited insight into the "why" behind issues
- Lack context of individual interactions or events
- Risk of data loss in case of collection/storage failure
- Fixed interval collection may miss critical details
- Excessive sampling can impact performance and costs.



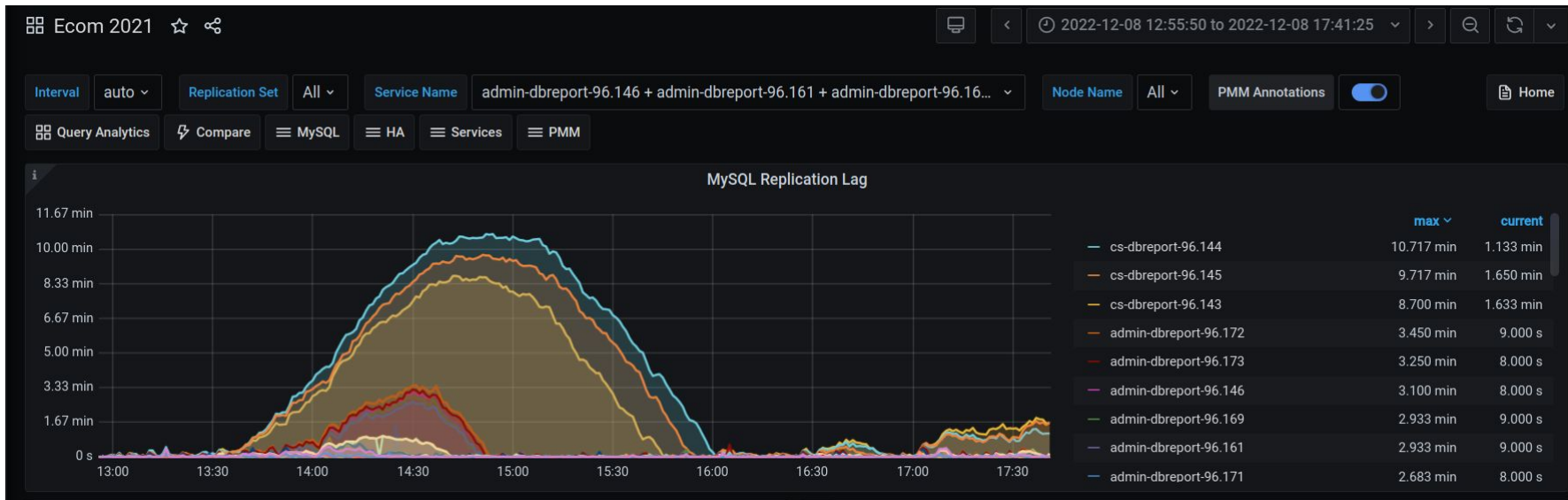
Metrics: Examples

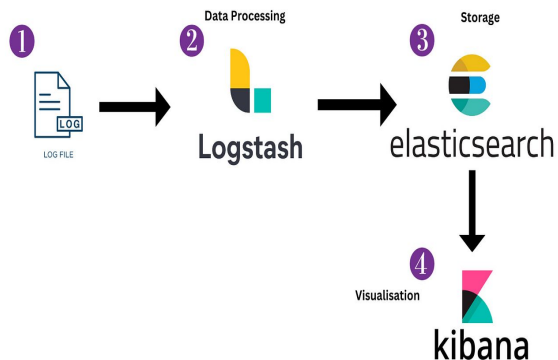
▼ Total Performance





Metrics: Examples





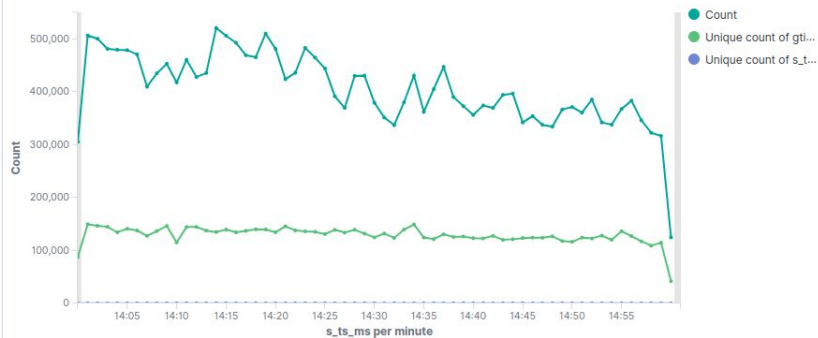
Advantages of Logs	Challenges of Logs
• Easy to generate, typically timestamp + plain text • Often require minimal integration by developers • Most platforms offer standardized logging frameworks • Human-readable, making them accessible • Provide granular insights for retrospective analysis	• Can generate large data volumes, leading to costs • Impact on application performance, especially without asynchronous logging • Retrospective use, not proactive • Persistence challenges in modern architectures • Risk of log loss in containers and auto-scaling environments



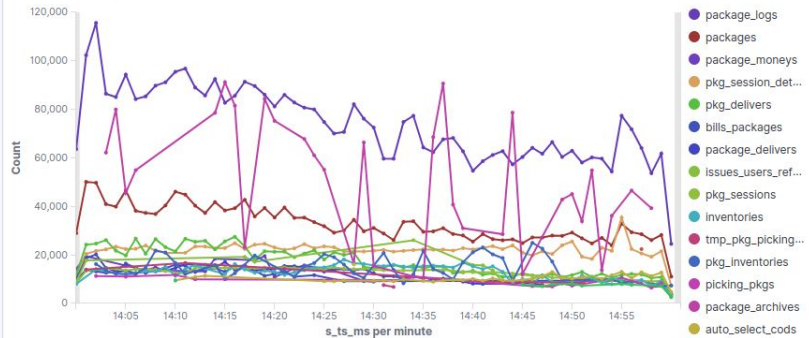
Logs: Examples

Kibana

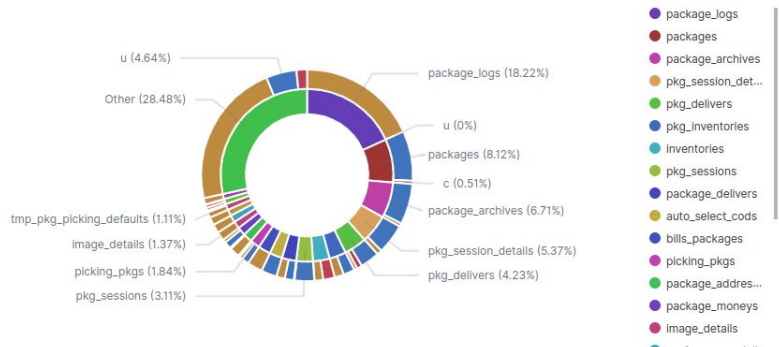
GHTK - binlog summary



GHTK - binlog top by affected



GHTK - binlog pie by table and op



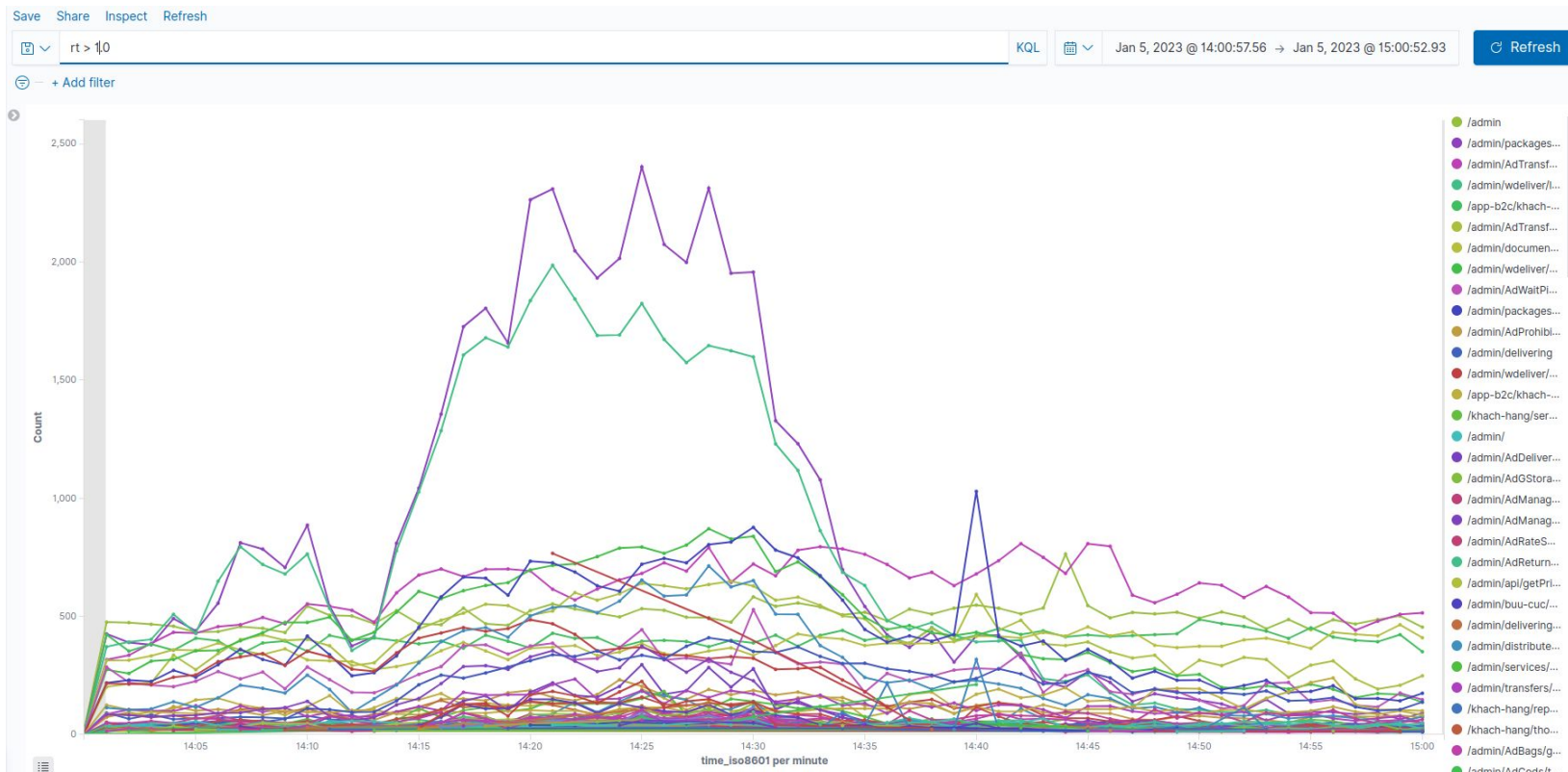
GHTK - binlog top by gtid

gtid	rows affected	start time	end time
13,321,870,204	7,858	Jan 5, 2023 @ 14:31:09.000	Jan 5, 2023 @ 14:31:13.000
13,317,896,623	6,001	Jan 5, 2023 @ 14:01:59.000	Jan 5, 2023 @ 14:02:05.000
13,319,604,558	5,710	Jan 5, 2023 @ 14:14:30.000	Jan 5, 2023 @ 14:14:35.000
13,323,825,311	5,171	Jan 5, 2023 @ 14:46:29.000	Jan 5, 2023 @ 14:46:31.000
13,318,555,158	5,101	Jan 5, 2023 @ 14:06:21.000	Jan 5, 2023 @ 14:06:49.000
13,321,466,432	5,022	Jan 5, 2023 @ 14:27:55.000	Jan 5, 2023 @ 14:28:06.000
13,321,765,700	4,848	Jan 5, 2023 @ 14:30:13.000	Jan 5, 2023 @ 14:30:22.000
13,324,010,660	4,831	Jan 5, 2023 @ 14:48:00.000	Jan 5, 2023 @ 14:48:00.000
13,320,008,652	4,765	Jan 5, 2023 @ 14:17:02.000	Jan 5, 2023 @ 14:17:31.000
13,322,181,484	4,735	Jan 5, 2023 @ 14:33:24.000	Jan 5, 2023 @ 14:33:32.000

Export: [Raw](#) [Formatted](#)



Logs: Examples

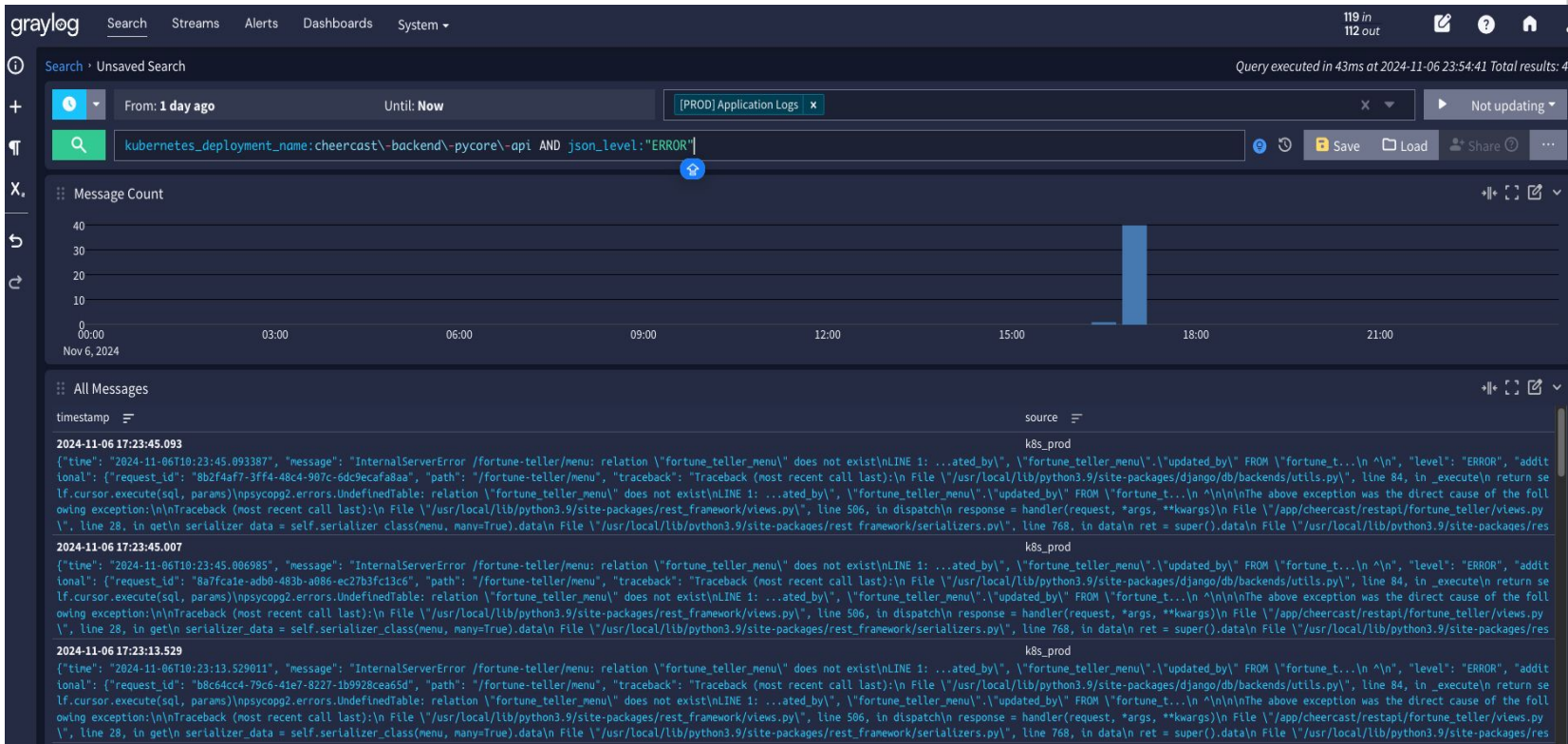


Kibana



Logs: Examples

Graylog





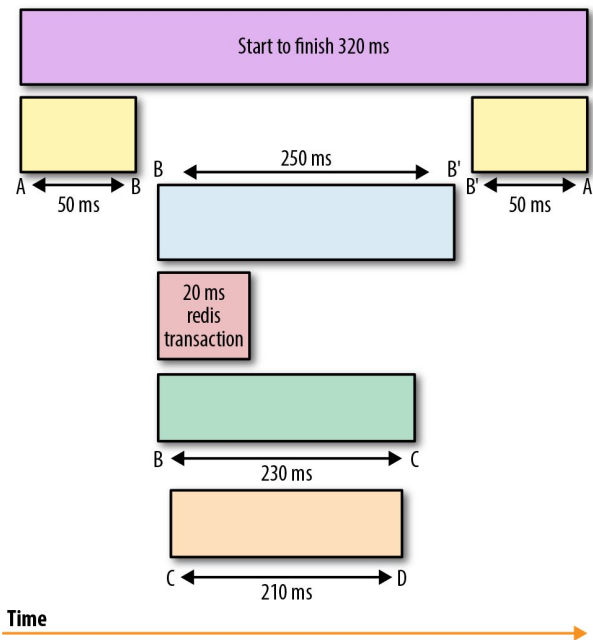
Logs: Examples

Graylog

Until: Now [PROD] Application Logs x

pycore\api AND LoggingRequestMiddleware

json_additional_request_path	count()	avg_response_time	max_response
/member/profile/details	1,183	0.0115706	0.163
/common/sys-data/get-by-code	1,109	0.0036727	0.095
/ai/get-chat-group-messages	1,015	0.0588483	0.336
/ai/get-chat-group	990	0.0566253	0.249
/internal/ai/user-details	988	0.0078209	0.049
/ai/get-avatars	839	0.1016901	0.461
/wallet/balance	759	0.0119473	0.107
/common/app-revision/filter-flag	624	0.0084199	0.171
/graphql/	511	0.0335362	0.411
/member/profile/update	426	0.0125164	0.115
/member/device/save	418	0.015512	0.149
/common/sys-data/public-setting	363	0.0092149	0.088
/togethers	204	0.0360588	0.239
/outfits	201	0.046806	0.31
/member/activity/page/togethers	200	0.027795	0.16
/ai/ai-user-latest-messages	139	0.0830432	0.287
/refresh-token	126	0.0170079	0.084
/fortune-teller/menu	113	0.0235221	0.079
/wallet/cash-flow	97	0.0114124	0.114
/ai/get-normal-avatars	92	0.0809022	0.282
/payment/coin-package	79	0.0122658	0.07
/common/v2/app-revision/filter-flag	75	0.0116	0.03
/register/confirm	73	0.0159041	0.06



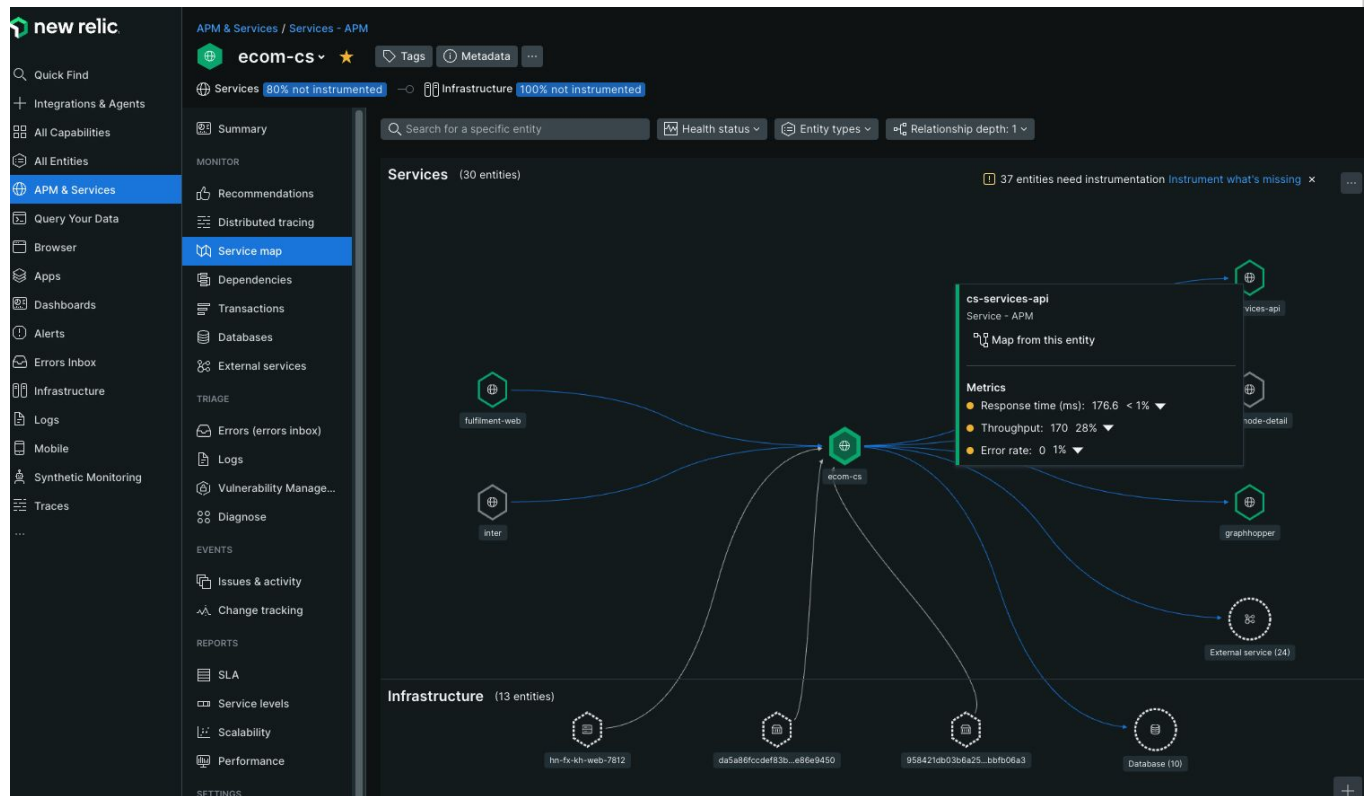
Advantages of Traces	Challenges of Traces
• Ideal for pinpointing issues within a service • Offers end-to-end visibility across multiple services • Identifies performance bottlenecks effectively • Aids debugging by recording request/response flows • Provides contextual insights into system behavior	• Limited ability to reveal long-term trends • Complex systems may yield diverse trace paths • Doesn't explain the cause of slow or failing spans (steps) • Adds overhead, potentially impacting system performance



Traces: Examples

Newrelic

<https://one.newrelic.com/>

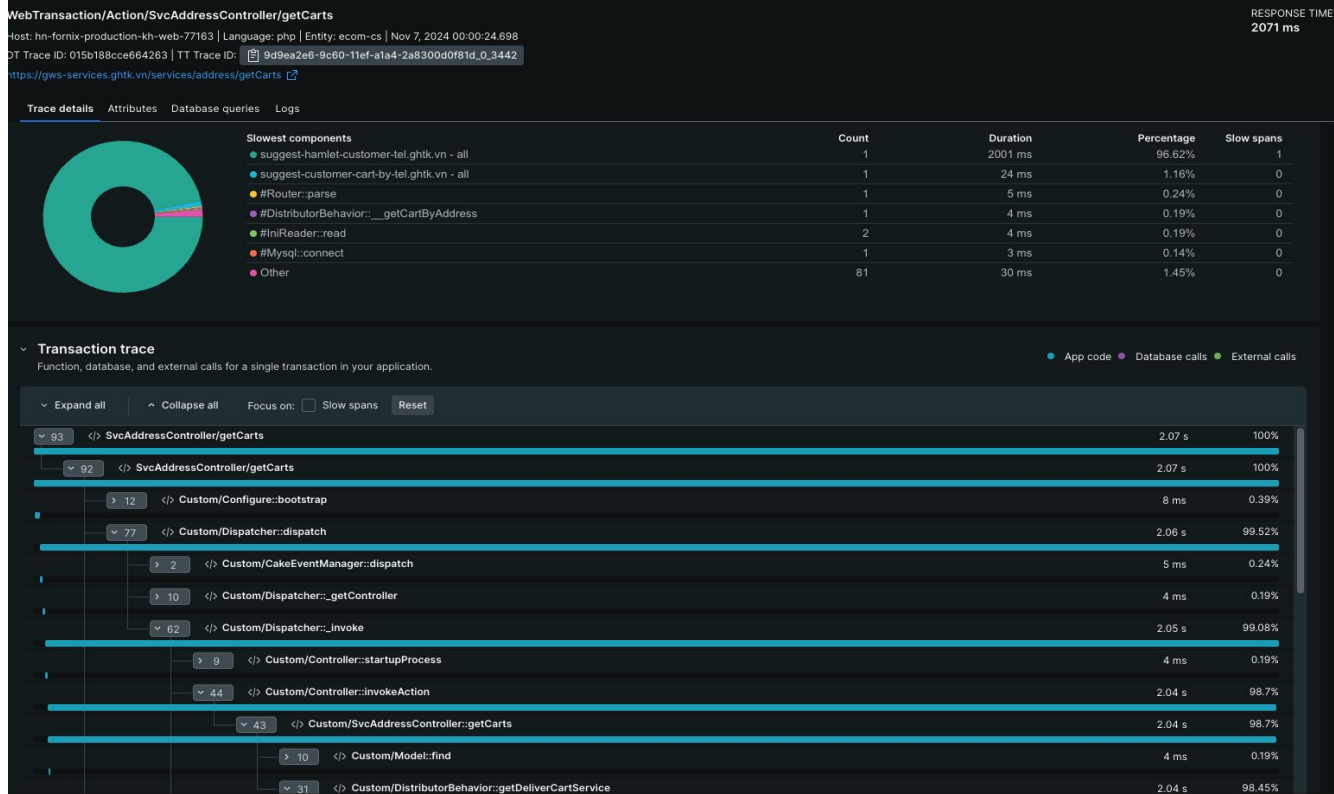




Traces: Examples

Newrelic

<https://one.newrelic.com/>





Traces: Examples

Sentry

<https://docs.sentry.io/product/sentry-basics/>

The screenshot shows the Sentry 'Issues' page. The top navigation bar includes filters for 'Unresolved' (82), 'For Review' (2), 'Regressed', 'Escalating', and 'Archived'. Below this, there are filters for 'frontend-javascript', 'All Envs', and '14D'. A search bar contains 'is:unresolved'. The main table lists several issues, including 'SyntaxError', 'Error', 'N+1 API Call', and 'TypeError'. Each issue entry includes a title, a brief description, a status (e.g., 'Unhandled'), and a bar chart representing the frequency of the error over time. The right side of the table has columns for 'EVENTS', 'USERS', and 'ASSIGNEE'.

The screenshot shows the Sentry 'Event Navigator' and 'Trace Navigator' for a specific event. The event ID is 'e7097add' and it occurred on 'May 15, 12:51'. The event title is 'process_commit_context_all_frames.get_commit_context' and the message is 'Organization integration does not exist'. The event has 18k events, 0 users, and a high priority. The 'Trace Navigator' shows a timeline of events, with the current event highlighted. The 'Event Highlights' section shows details about the event, including 'handled', 'level', 'release', 'environment', 'url', and 'transaction'. The 'Message' section shows the error message 'process_commit_context_all_frames.get_commit_context_all_frames.unknown_error'. The 'Stack Trace' section shows the code location where the error occurred, specifically in 'sentry/integrations/github/integration.py' at line 164. The 'IntegrationError' section shows the error message 'Organization Integration does not exist'. The right side of the page shows the 'Assigned To' field (No one), the 'Last 24 Hours' graph (173 events), the 'Last 30 Days' graph (3.6k events), and the 'Last Seen' field (12 minutes ago in release 660ec70b6dbt).



Traces: Examples

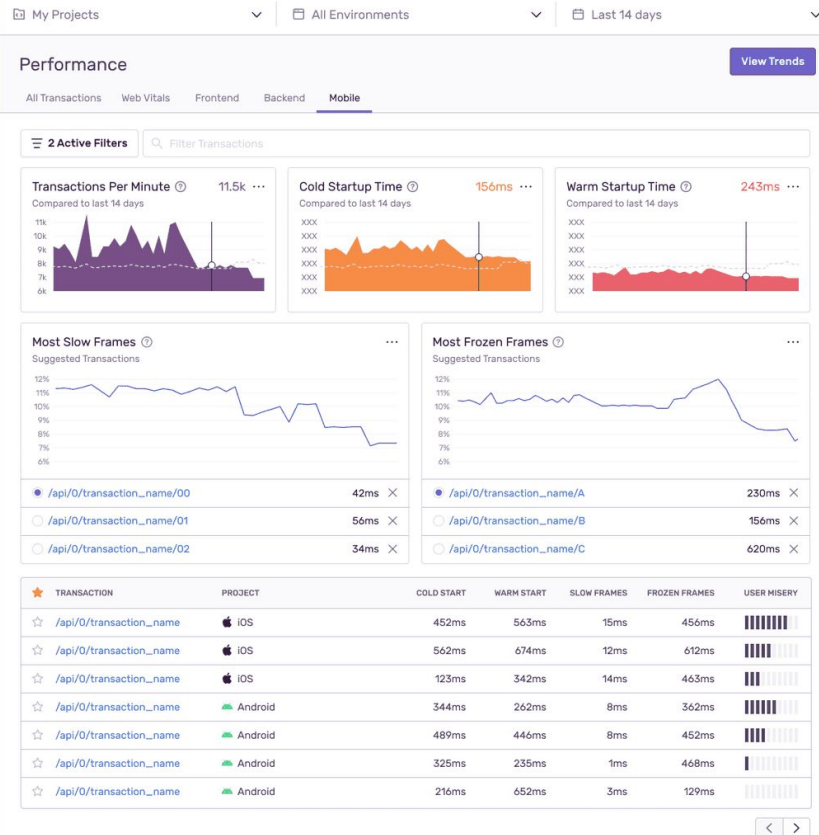
Sentry

- Measures cold starts and warm starts.
- On iOS, Apple recommends app take at most 400ms to render the first frame
- On Android, the Google Play console warns when a cold start takes longer than 5 seconds or a warm start longer than 2 seconds.

Empower Plant
Jane Schmidt

- Projects
- Issues
- Performance
- Releases
- User Feedback
- Alerts
- Discover
- Dashboards
- Activity
- Stats
- Settings

Help
What's new
Collapse

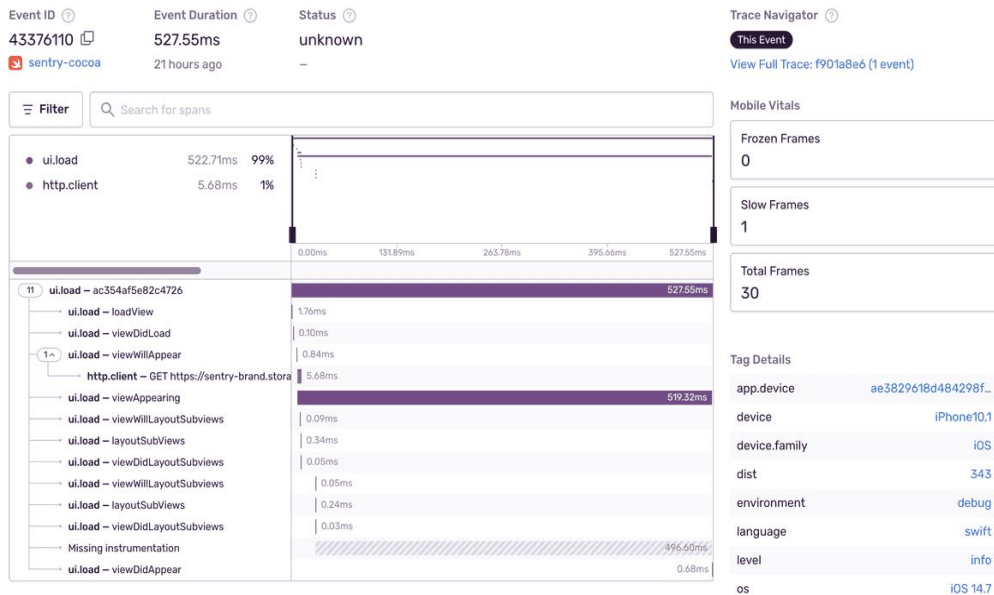




Traces: Examples

Sentry

Slow and Frozen Frames





Observability tools

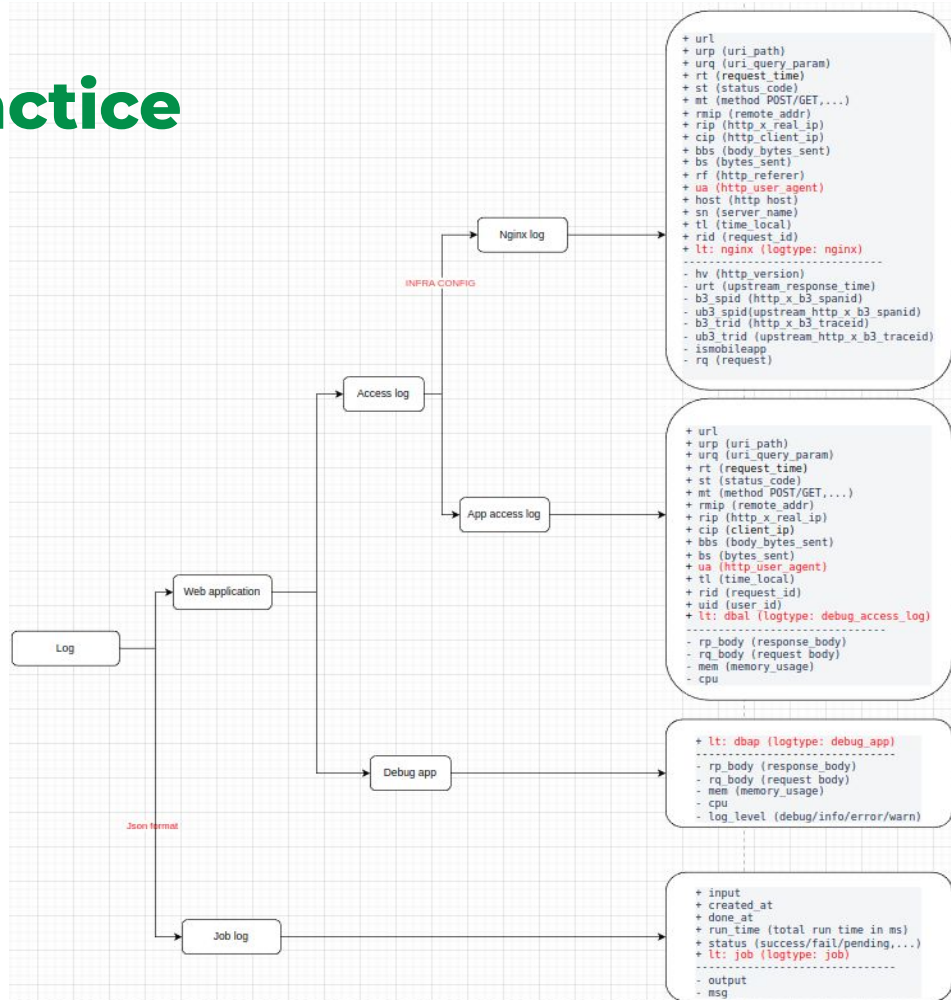
1. **Prometheus:** A leading open-source monitoring and alerting toolkit known for its scalability and support for multi-dimensional data collection.
2. **Grafana:** A visualization and dashboarding platform often used with Prometheus, providing rich insights into system performance.
3. **Jaeger:** An open-source distributed tracing system for monitoring and troubleshooting microservices-based architectures.
4. **Elasticsearch:** A search and analytics engine that, when paired with Kibana and Beats, forms the [ELK Stack](#) for log management and analysis.
5. **Datadog:** A cloud-based observability platform that integrates logs, metrics, and traces, providing end-to-end visibility.
6. **New Relic:** Offers application performance monitoring (APM) and infrastructure monitoring solutions to track and optimize application performance.



Logging Best Practice

Quy chuẩn logging:

https://docs.google.com/document/d/1iANG_KHuxVLerrhv1HCen6_cmztQN153dAqC83fjGek/edit?tab=t.0



Question?